

Django Documentation

IPL First Innings Score Predictor

1. Introduction

Project Title

IPL First Innings Score Predictor

Project Description

This project is a Django-based web application integrated with a Machine Learning model to predict the final first innings score in an IPL cricket match.

The application allows users to enter live match details such as batting team, bowling team, current score, wickets, and overs. Based on these inputs, the integrated Machine Learning model predicts the estimated final innings score.

The project demonstrates the integration of:

- Django web framework
- Scikit-learn Machine Learning model
- Frontend user interface
- Backend prediction system

2. Objectives

The main objectives of the project are:

- Build a dynamic web application using Django
 - Integrate Machine Learning prediction into a website
 - Create a user-friendly interface for cricket score prediction
 - Handle user input using Django forms
 - Display real-time prediction results
-

3. What is Django?

Django is a high-level Python web framework used for building secure, scalable, and maintainable web applications.

Features of Django

- Fast development
 - Built-in admin panel
 - URL routing system
 - Secure authentication
 - Template engine
 - ORM (Object Relational Mapping)
 - Scalability and reusability
-

4. Technologies Used

Technology	Purpose
Python	Programming language
Django	Web framework
HTML	Frontend structure
CSS	Styling and UI
SQLite	Database management
Scikit-learn	Machine Learning model
JSON	Model storage

5. Django Project Structure

Main Components

manage.py

Used for managing Django commands such as:

- Running server
- Database migrations
- App management

predictor App

Handles the prediction functionality.

Templates

Contains HTML frontend pages.

Static Files

Contains CSS and static resources.

6. Folder Structure

```
IPL Score Predictor
|
|— manage.py
|— db.sqlite3
|— requirements.txt
|
|— predictor/
|   |— admin.py
|   |— apps.py
|   |— forms.py
|   |— models.py
|   |— services.py
|   |— urls.py
|   |— views.py
|   |— templates/
|   |— ml/
|
|— ipl_score_predictor/
|   |— settings.py
|   |— urls.py
|   |— wsgi.py
|   |— asgi.py
```

7. Django Architecture

Django follows the MVT architecture:

MVT Architecture

Model

Handles database operations.

View

Contains business logic and handles requests.

Template

Handles frontend user interface.

8. Request Flow in Django

User Request → URL → View → Machine Learning Logic → Template Response

9. URL Configuration

Django uses URL routing to connect webpages with backend functions.

Example:

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home, name='home'),
]
```

Purpose

- Connects URLs to views
 - Handles page navigation
 - Organizes application routes
-

10. Django Views

Views contain the main application logic.

In this project, the view:

- Accepts user input
- Processes form data
- Calls prediction services
- Sends prediction results to frontend

Example:

```
from django.shortcuts import render

def home(request):
    return render(request, 'predictor/home.html')
```

11. Django Templates

Templates are HTML files used to display frontend content.

Features Used

- Forms
- Dynamic data rendering
- Responsive UI design

Example:

```
<h1>IPL First Innings Score Predictor</h1>
```

12. Django Forms

Forms are used to collect user input.

Inputs Collected

- Batting team
- Bowling team
- Current score
- Overs completed
- Wickets lost
- Runs in last 5 overs
- Wickets in last 5 overs

Example:

```
from django import forms

class PredictionForm(forms.Form):
    score = forms.IntegerField()
```

13. Machine Learning Integration in Django

The Machine Learning model is integrated into Django using service modules.

Workflow

1. User enters match data
2. Django form validates input
3. View sends data to prediction service
4. Scikit-learn model predicts score
5. Result returned to template

14. Services Module

The services.py file handles:

- Prediction logic
- Data transformation
- Model loading
- Result generation

Advantages:

- Better code organization
- Reusable logic

- Cleaner views
-

15. Database Usage

The project uses SQLite database.

Purpose

- Store prediction history
- Manage Django configurations
- Handle application data

Database file:

```
db.sqlite3
```

16. Static Files

Static files include:

- CSS files
- Images
- Frontend assets

Purpose:

- Improve UI appearance
 - Enhance user experience
 - Provide responsive design
-

17. Django Settings

The settings.py file contains:

- Installed apps
- Database configuration
- Template settings
- Static file settings
- Security settings

Example:

```
INSTALLED_APPS = [  
    'predictor',  
]
```

18. WSGI and Deployment

The project uses WSGI for deployment.

WSGI Purpose

WSGI acts as a bridge between:

- Django application
- Web server

Deployment platform used:

- PythonAnywhere

19. Features of the Application

- Real-time IPL score prediction
- User-friendly interface
- Django backend integration
- Machine Learning prediction system
- Responsive frontend design
- Prediction output display

20. Advantages of Django in This Project

- Rapid development
 - Clean architecture
 - Built-in security
 - Easy routing system
 - Scalable backend
 - Easy integration with Machine Learning
-

21. Limitations

- Limited to IPL first innings prediction
 - Basic database implementation
 - Requires internet for deployment access
 - Prediction accuracy depends on ML model quality
-

22. Future Improvements

Future enhancements may include:

- User authentication system
 - Live IPL API integration
 - Advanced ML algorithms
 - Match analytics dashboard
 - Graphical statistics visualization
 - Mobile responsiveness improvements
-

23. Testing

The application was tested for:

- Form validation
 - Prediction accuracy
 - URL routing
 - Frontend responsiveness
 - Django server functionality
-

24. Deployment Process

The project was deployed using PythonAnywhere.

Deployment Steps

1. Upload project files
2. Create virtual environment
3. Install dependencies
4. Configure WSGI file
5. Set static file paths

25. How to Run the Project

Step 1

Install dependencies:

```
pip install -r requirements.txt
```

Step 2

Run migrations:

```
python manage.py migrate
```

Step 3

Start Django server:

```
python manage.py runserver
```

Step 4

Open browser:

```
http://127.0.0.1:8000/
```

26. Sample Output

The user enters match details and receives predicted innings score.

Example:

Input Score	Overs	Wickets	Predicted Score
120	15	4	178

27. Conclusion

The IPL First Innings Score Predictor successfully demonstrates the use of Django for building Machine Learning-powered web applications.

The project combines backend development, frontend design, and Machine Learning integration into a single deployable platform.

It also highlights how Django can efficiently manage routing, templates, forms, and prediction workflows.

28. References

- Django Official Documentation
 - Python Documentation
 - Scikit-learn Documentation
 - PythonAnywhere Documentation
-

29. Requirements

```
Python
Django
Scikit-learn
Pandas
NumPy
SQLite
```

30. Author Information

Project Name: IPL First Innings Score Predictor Framework: Django Category: Machine Learning Web
Application Deployment Platform: PythonAnywhere